

Homework 1

Guess: Left

Part 1

1

```
present_party <- function(target = 9, rule = c("range", "abs"), p = 0.5) {  
  rule <- match.arg(rule)  
  pos <- 0  
  max_pos <- 0  
  min_pos <- 0  
  n_steps <- 0  
  
  repeat {  
    step <- if (runif(1) < p) 1 else -1  
    pos <- pos + step  
    n_steps <- n_steps + 1  
    max_pos <- max(max_pos, pos)  
    min_pos <- min(min_pos, pos)  
  
    done <- if (rule == "range") (max_pos - min_pos) >= target else abs(pos) >= target  
    if (done) break  
  }  
  
  list(n_steps = n_steps, final_position = pos,  
       max_reached = max_pos, min_reached = min_pos)  
}  
set.seed(123)  
present_party()
```

```
$n_steps  
[1] 26
```

```
$final_position  
[1] -8
```

```
$max_reached  
[1] 1
```

```
$min_reached  
[1] -8
```

2

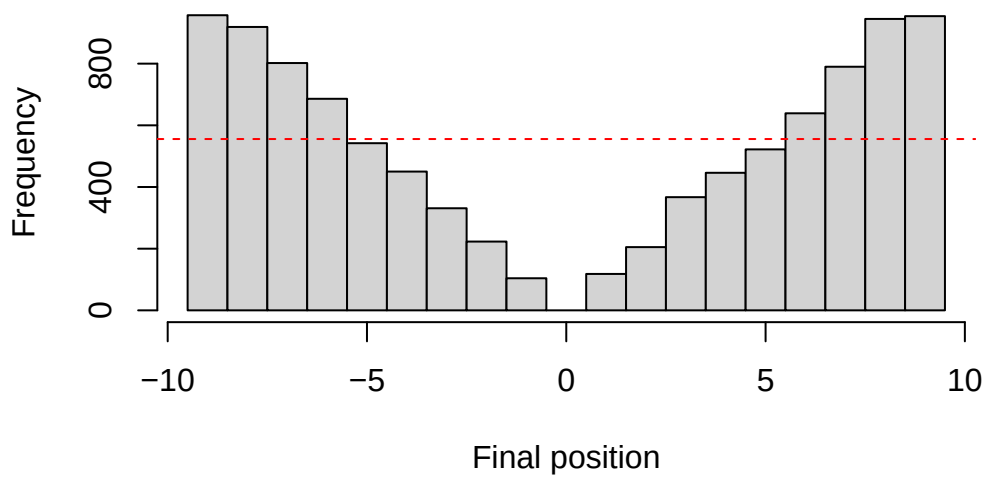
This function is correct because it records the number of steps took and the minimum value reached. Also for the round table, I assume that the starting position is 0 (The host), and make sure that sum of the positive steps (move right) and absolute value of negative step (move left) must be equal to 9.

Part 2

3 and 4

```
set.seed(2026)  
n_sim <- 10000  
sims <- do.call(rbind, lapply(seq_len(n_sim), function(i)  
  as.data.frame(present_party())))  
  
hist(sims$final_position,  
     breaks = seq(-9.5, 9.5, by = 1),  
     main = "Histogram of final position", xlab = "Final position")  
abline(h = nrow(sims) / 18, col = "red", lty = 2) # equally-likely reference
```

Histogram of final position



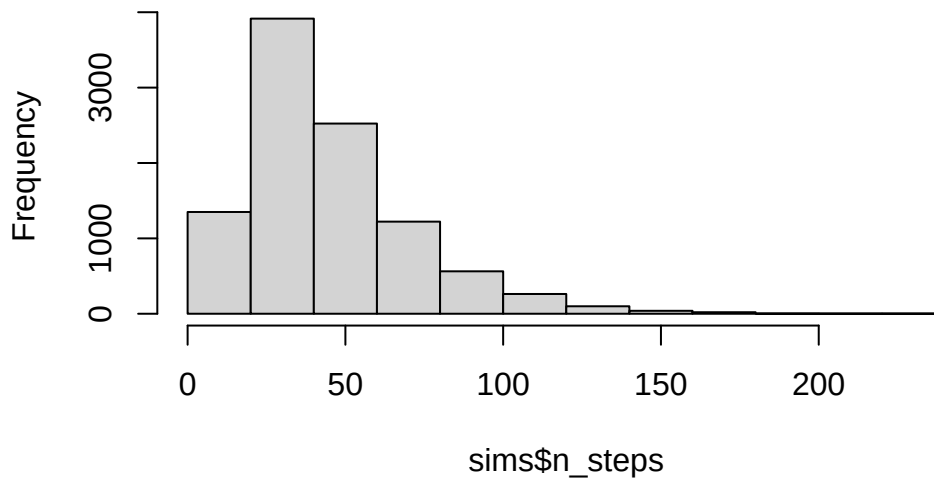
5

```
mean(sims$n_steps)
```

```
[1] 45.0647
```

```
hist(sims$n_steps)
```

Histogram of sims\$n_steps



The average number of passes per party is 45.06. The distribution is heavily right skewed.

6

I should pick the chair that is the farthest from the host. My guess is wrong. Ending at the farthest seat (+9 or -9 in my simulation) is the most likely outcome because it only needs the walk to stay on one side of 0. Ending near 0 needs the walk to sweep all the way to the far side first, which is unlikely.

7

```
positions <- c(-9:-1, 1:9)
obs <- table(factor(sims$final_position, levels = positions))

# Test 1: H0 = all 18 positions equally likely
t_uniform <- chisq.test(obs, p = rep(1/18, 18))

# Test 2: H0 = theoretical P(final = +/-k) = k/90
p_theory <- abs(positions) / 90
t_theory <- chisq.test(obs, p = p_theory)

# Test 3: directly compare +/-9 vs +/-1 (conditional binomial test)
n9 <- sum(obs[c("-9", "9")]); n1 <- sum(obs[c("-1", "1")])
t_ends <- binom.test(n9, n9 + n1, p = 0.5, alternative = "greater")

print(t_uniform); print(t_theory); print(t_ends)
```

Chi-squared test for given probabilities

```
data:  obs
X-squared = 2668.7, df = 17, p-value < 2.2e-16
```

Chi-squared test for given probabilities

```
data:  obs
X-squared = 19.251, df = 17, p-value = 0.3143
```

Exact binomial test

```
data:  n9 and n9 + n1
number of successes = 1911, number of trials = 2133, p-value < 2.2e-16
alternative hypothesis: true probability of success is greater than 0.5
95 percent confidence interval:
 0.8844046 1.0000000
sample estimates:
probability of success
      0.8959212
```

```
cat("Ratio (+/-9)/(+/-1):", round(n9/n1, 2), "\n")
```

Ratio (+/-9)/(+/-1): 8.61

Test 1 ($p < 2.2e-16$): the final positions are not equally likely.

Test 2 ($p = 0.31$): the data fit the $k/90$ formula.

Test 3 ($p < 2.2e-16$): +/-9 is far more likely than +/-1. About 90% of the walks ending at +/-9 or +/-1 ended at +/-9, which matches the theoretical 9/10. Ratio 8.61: +/-9 happened about 9 times as often as +/-1, as the theory predicts.

Part 3

8

```
seat_probs <- function(p, n_sim = 10000) {
  set.seed(2026)
  seat <- replicate(n_sim, present_party(p = p)$final_position) %% 10
  as.numeric(table(factor(seat, levels = 1:9))) / n_sim
}
biases <- c(0.5, 0.4, 0.2, 0.1)

# for loop
probs_loop <- matrix(NA, length(biases), 9)
for (i in seq_along(biases)) probs_loop[i, ] <- seat_probs(biases[i])

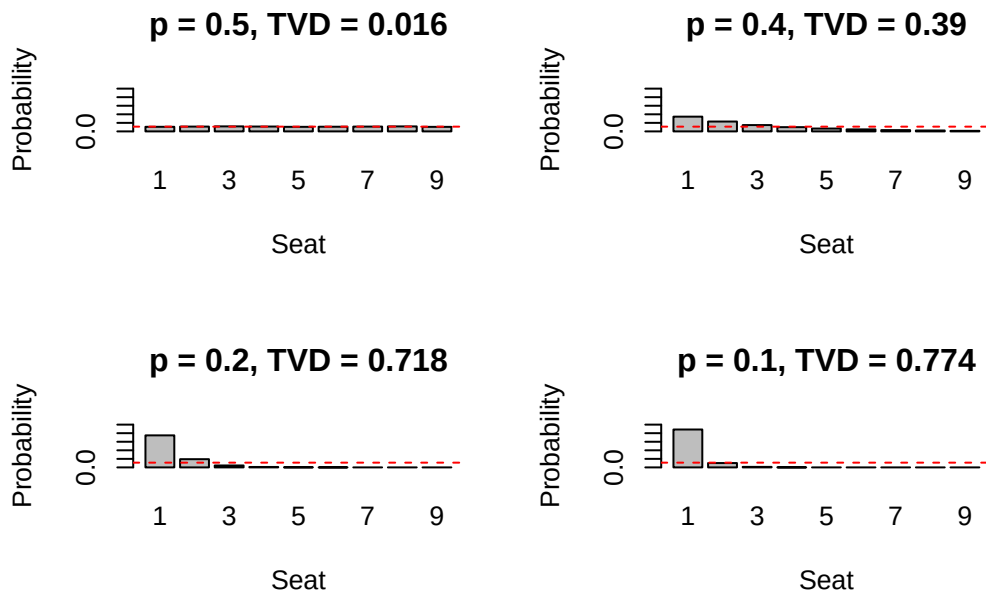
# functional
probs <- t(sapply(biases, seat_probs))
```

```
stopifnot(all.equal(probs_loop, probs))

result <- data.frame(p = biases,
                    TVD = round(0.5 * rowSums(abs(probs - 1/9)), 3),
                    round(probs, 3))
names(result)[3:11] <- paste0("seat", 1:9)
print(result)
```

	p	TVD	seat1	seat2	seat3	seat4	seat5	seat6	seat7	seat8	seat9
1	0.5	0.016	0.108	0.112	0.117	0.113	0.106	0.109	0.112	0.117	0.106
2	0.4	0.390	0.345	0.231	0.148	0.099	0.067	0.047	0.030	0.021	0.013
3	0.2	0.718	0.749	0.192	0.045	0.010	0.003	0.001	0.000	0.000	0.000
4	0.1	0.774	0.886	0.101	0.012	0.001	0.000	0.000	0.000	0.000	0.000

```
par(mfrow = c(2, 2))
for (i in seq_along(biases)) {
  barplot(probs[i, ], names.arg = 1:9, ylim = c(0, 1),
          main = paste0("p = ", biases[i], ", TVD = ", result$TVD[i]),
          xlab = "Seat", ylab = "Probability")
  abline(h = 1/9, col = "red", lty = 2)
}
```



```

sweep_loop <- function() {
  out <- matrix(NA, length(biases), 9)
  for (i in seq_along(biases)) out[i, ] <- seat_probs(biases[i])
  out
}
sweep_fun <- function() t(sapply(biases, seat_probs))

times <- t(replicate(10, c(loop      = system.time(sweep_loop())["elapsed"],
                           functional = system.time(sweep_fun())["elapsed"])))
apply(times, 2, median)

```

```

      loop.elapsed functional.elapsed
      1.1335          1.1275

```

```
t.test(times[, 1], times[, 2], paired = TRUE)
```

Paired t-test

```

data: times[, 1] and times[, 2]
t = 2.0835, df = 9, p-value = 0.06689
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -0.0007547692  0.0183547692
sample estimates:
mean difference
      0.0088

```

I think it's going to be similar: `sapply()` is itself a loop over the 4 values of `p`, and each one runs 10,000 walks step by step in R. That work should dwarf the tiny cost of looping 4 times, and the `for` loop already fills a preallocated matrix, which avoids the usual slow-loop trap.

The `t` test shows that there is no significant evidence to reject the null hypothesis that the difference is 0.

10

I'd give the function. The loop can go wrong when you change the game to 4 people for example, so `seat_probs()` returns 3 seats, but forgets the hard-coded 9 in `matrix()`. The loop then fills each row by repeating the 3 values: 0.2 0.3 0.5 0.2 0.3 0.5 0.2 0.3 0.5. The table shows 9 seats with plausible numbers, and nothing flags it. Also, it **can fail far from the cause**. If the new game has more than 9 seats, the loop does stop, but only with **number of items to replace is not a multiple of replacement length**, which doesn't point at the real mistake.

11

```
# How biased must the coin be before your seat matters?
# Exact seat probabilities for a biased coin (no simulation noise),
# using the gambler's-ruin formula.

# P(walk hits b before a | starts at x), with a < x < b and P(step +1) = p
ruin_up <- function(x, a, b, p) {
  if (abs(p - 0.5) < 1e-12) return((x - a) / (b - a)) # fair coin
  r <- (1 - p) / p
  (1 - r^(x - a)) / (1 - r^(b - a))
}

# Exact probability that each seat (1..target) gets the present last.
# Ending at +k: first reach k - target before k, then climb from there
#           to k before dropping to k - target - 1.
# Ending at -k: mirror image.
exact_seats <- function(p, target = 9) {
  k <- 1:target
  up <- vapply(k, function(k) (1 - ruin_up(0, k - target, k, p)) *
    ruin_up(k - target, k - target - 1, k, p), numeric(1))
  down <- vapply(k, function(k) ruin_up(0, -k, target - k, p) *
    (1 - ruin_up(target - k, -k, target - k + 1, p)), numeric(1))
  seat <- c(k, -k) %% (target + 1) # -k is the same seat as target + 1 - k
  as.numeric(tapply(c(up, down), factor(seat, levels = 1:target), sum))
}

# Unfairness = total variation distance from fair (1/9 per seat)
tvd <- function(p) 0.5 * sum(abs(exact_seats(p) - 1/9))

# Games needed for a chi-square test (alpha = 0.05) to detect the bias with 80% power
```

```

games_to_detect <- function(p, alpha = 0.05, power = 0.80, df = 8) {
  if (p == 0.5) return(NA)
  s <- exact_seats(p)
  w2 <- sum((s - 1/9)^2 / (1/9)) # effect size (Cohen's w^2)
  crit <- qchisq(1 - alpha, df)
  lam <- uniroot(function(l) pchisq(crit, df, ncp = l) - (1 - power), c(0, 1000))$root
  ceiling(lam / w2)
}

# Sanity checks
stopifnot(all.equal(exact_seats(0.5), rep(1/9, 9))) # fair coin -> uniform
stopifnot(all.equal(sum(exact_seats(0.4)), 1)) # probabilities sum to 1

# ---- Table ----
ps <- c(0.5, 0.499, 0.495, 0.49, 0.48, 0.47, 0.46, 0.45, 0.42, 0.4)
result <- data.frame(
  p = ps,
  TVD = round(vapply(ps, tvd, numeric(1)), 3),
  best_seat = round(vapply(ps, function(p) max(exact_seats(p)), numeric(1)), 3),
  worst_seat = round(vapply(ps, function(p) min(exact_seats(p)), numeric(1)), 3),
  best_over_worst = round(vapply(ps, function(p) { s <- exact_seats(p); max(s) / min(s) }, numeric(1)), 3),
  games_to_detect = vapply(ps, games_to_detect, numeric(1))
)
print(result)

```

	p	TVD	best_seat	worst_seat	best_over_worst	games_to_detect
1	0.500	0.000	0.111	0.111	1.00	NA
2	0.499	0.004	0.113	0.109	1.03	140836
3	0.495	0.022	0.120	0.102	1.17	5637
4	0.490	0.045	0.130	0.094	1.38	1412
5	0.480	0.089	0.150	0.079	1.90	355
6	0.470	0.133	0.171	0.066	2.61	160
7	0.460	0.175	0.194	0.054	3.61	91
8	0.450	0.216	0.218	0.044	4.98	60
9	0.420	0.323	0.292	0.022	13.23	25
10	0.400	0.389	0.342	0.013	25.63	17

```

# Is unfairness linear in the bias? (constant ratio -> linear -> no "safe zone")
cat("\nTVD / |p - 0.5|:\n")

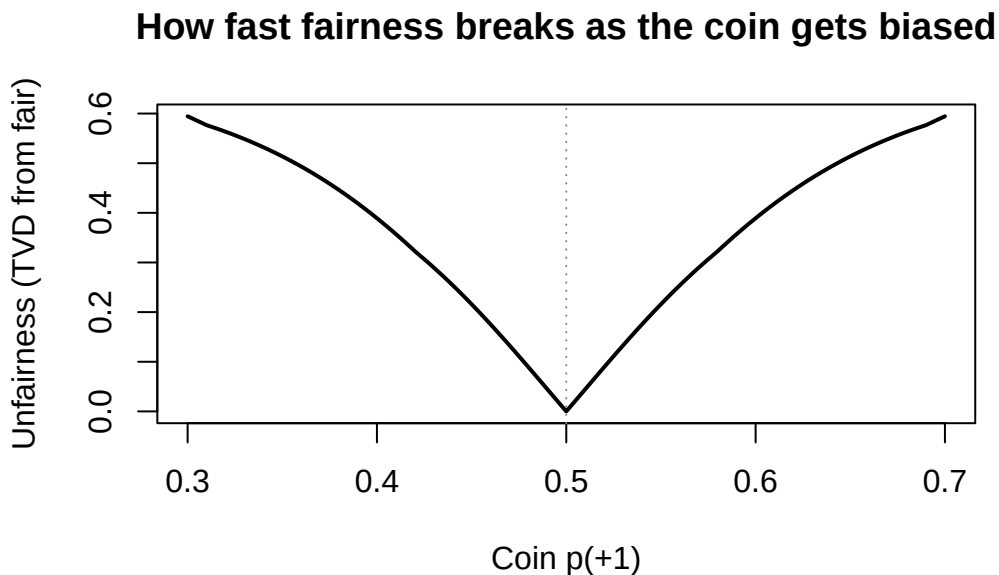
```

TVD / $|p - 0.5|$:

```
print(round(sapply(c(0.499, 0.49, 0.45), function(p) tvd(p) / abs(p - 0.5)), 2))
```

```
[1] 4.45 4.46 4.32
```

```
# ---- Plot: unfairness vs bias ----  
p_grid <- seq(0.3, 0.7, by = 0.0025)  
plot(p_grid, vapply(p_grid, tvd, numeric(1)), type = "l", lwd = 2,  
      xlab = "Coin p(+1)", ylab = "Unfairness (TVD from fair)",  
      main = "How fast fairness breaks as the coin gets biased")  
abline(v = 0.5, lty = 3, col = "grey50")
```



I computed the exact seat probabilities for each p using the gambler's-ruin formula, so this table has no simulation noise. It matches the earlier simulations, for example TVD 0.389 exact against 0.390 simulated at $p = 0.4$. Also, you can see that the line goes linearly, so a tiny bias has effect. So I don't think there's a safe zone.

AI disclosure

I translated the question into this prompt and asked AI for the code:

write a function in R that conduct the follow:

1. Start from position 0

2. Conduct simulation. For each simulation, there is 50% to move to -1, and 50% to move to 1.
3. Conduct the next simulation. If the first simulation move to -1, then there is 50% to move to -2, and 50% move back to 0, vice versa.
4. Conduct as many simulation as needed. Stop the simulation when the absolute value of the position equals to 9. For example, The max reached to negative is -3 and max reached to positive is 6, or max reached to positive is 9 and never reach negative tor max reached to negative is -9 and never reach positive.
5. Record the number of simulation, and the final position after the last simulation.

For question 8, I asked AI to change the code into loop/function.

For question 11, I was not sure how to do it, so I asked AI to simulate the study for different p with small value and give me the code.